

Q1. DBMS vs. Traditional File System

- File System: Maintains data in separate and disorganized files. This causes data redundancy, inconsistencies, and absence of concurrency.
- DBMS: Centralized computer software system maintaining one pool of data. It ensures data integrity, enables safe concurrent access, and keeps data definition separate from program code.
- Concrete Example and Explanation
- Example: An aircraft flight booking application.
- Explanation: If using the file system, two agents might concurrently open flights.txt and sell the last ticket to two passengers. The DBMS employs concurrency control through locking in order to carry out bookings in series, ensuring that in case the system shuts down while processing payment, all actions would be rolled back.

Q2. Strong vs. Weak Entities

- Distinction: A strong entity can stand alone; that is, it has its own primary key which is used to identify records. Conversely, a weak entity is totally dependent on a parent or strong entity, as it does not have any attribute that can be used to identify it.
- Formation of Primary Key: This is a combination of two keys. One key is that of the parent entity, and the other is that of the weak entity.
- Identifying Relationship: This is the relationship between the weak and the parent or strong entity.

Q3. Modeling Student Phone Numbers

It must be designed as a multivalued attribute.

Reason

- One person can have several telephone numbers, thus meeting the criterion of a multivalued attribute (one value, several values, or no value at all). Designing the attribute as a simple attribute will make it difficult to store several numbers in one cell, as this will violate the principle of atomicity. The attribute is not a composite attribute since a telephone number cannot be decomposed further.

Q4. Candidate Keys vs. Super Keys

The Difference

- Super Key: Is any collection of attribute(s) which uniquely identify a tuple in a relation. Super key may consist of additional unnecessary columns.
- Candidate Key: Is a minimal super key. Candidate key does not have any unnecessary attributes. If we drop one attribute then it will not be a super key anymore.

Example for Relation: Student(roll no, email, name, branch)

- Candidate Keys: {roll no}, {email}
- Super Key: {roll no, name}, {email, branch}

Q5. Enrollment Table Insertion Violation

- Constraint Violated: Foreign Key Constraint (Referential Integrity).

- Protection Offered: It avoids the creation of orphaned records. It guarantees that the database will not refer to a non-existing student record.
- What Should be Done: The DBMS should abort the insertion of this record and generate an error message indicating the violation of integrity.

Q6. Mapping Many-to-Many (M:N) Relationships

- Additional Structure: A new junction table (also called a bridge or associative table) must be created.
- Primary Key: A composite key consisting of the primary keys of both participating entity tables.
- Why it can't be a column: If you added a Course_ID column inside a Student table, you would have to either stuff multiple values into a single cell (violating database rules) or duplicate the entire student's row for every course they take (causing massive data redundancy). A separate table resolves this cleanly.

Part A – Entity and Relationship Identification

1. Entities and Key Attributes

- Customer: **CustomerID** (Key), Name, Email, Phone, Address
- Restaurant: **RestaurantID** (Key), Name, Location, CuisineType
- Order: **OrderID** (Key), OrderDate, TotalAmount, Status
- MenuItem: **ItemID** (Key), Name, Price, Description

2. Relationships and Cardinality

- Customer to Order (1:N): One customer can place multiple orders over time, but each specific order belongs to only one customer.
- Restaurant to Order (1:N): A restaurant can receive many different orders, but any single order is placed at exactly one restaurant.
- Order to MenuItem (M:N): An order can contain multiple different menu items, and a single menu item can be a part of many different orders.
- Restaurant to MenuItem (1:N): A restaurant can offer many different items on its menu, but each specific menu item belongs to that unique restaurant.

Part B – Attribute Classification

- Multivalued Attribute: **Customer.Phone** (or **Customer.SavedAddresses**)
 - Reason: It qualifies because a single customer can have more than one active phone number (e.g., primary and secondary) or multiple saved delivery locations linked to their account.
- Composite Attribute: **Customer.Address**
 - Reason: It qualifies because a complete delivery address is not a single atomic value; it can be broken down into smaller, independent sub-components like

HouseNo, Street, City, and ZipCode.

Part C – Relational Schema Design

To represent the M:N relationship between **Order** and **MenuItem**, a junction table (e.g., **OrderDetails**) is required to track which items were bought in which order, and how many of each were requested.

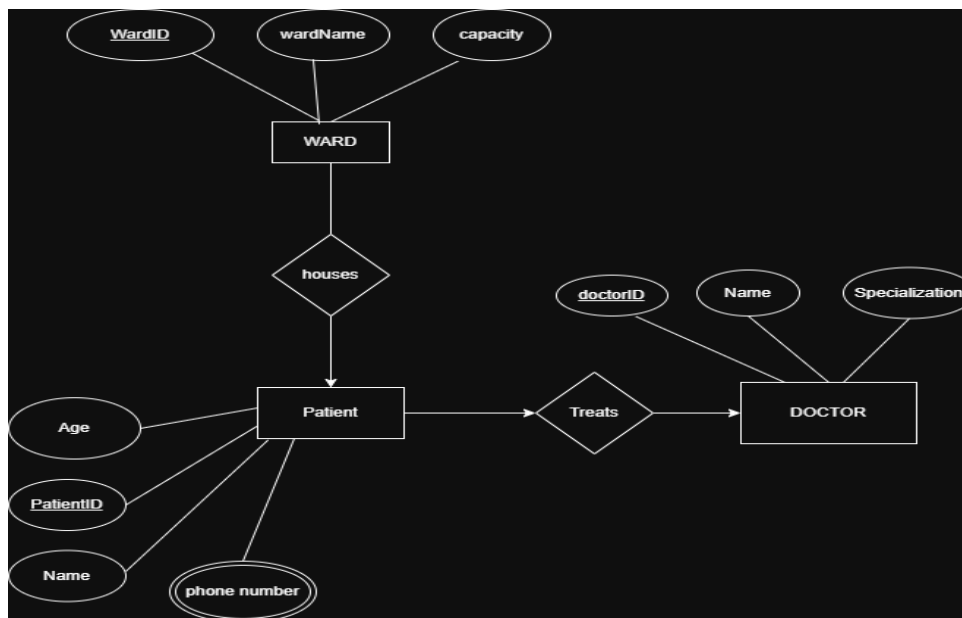
Schema Notation:

OrderDetails(OrderID, ItemID, Quantity)

Key Specifications:

- Primary Key: Composite key consisting of (OrderID, ItemID).
- Foreign Keys: * OrderID references the Order table.
 - ItemID references the MenuItem table.
- Additional Attributes: Quantity (tracks how many units of that specific item were ordered).

Part III: ER Diagram Design Question 1: Hospital ER Diagram



Question 2: Design Justification

The Doctor–Patient relationship is modeled as M:N because a single doctor can manage multiple patients, while a patient can be treated by a team of different specialists during their hospital stay. In a physical database schema, this M:N relationship resolves into an associative entity (junction table) typically named **Treatment** or **Appointment**. The composite primary

key for this associative entity would consist of the foreign keys from both parent entities: (**DoctorID**, **PatientID**).

Regarding participation constraints, the relationship between **Ward** and **Patient** features total participation on the Patient side because a patient cannot legally be hospitalized in this system without being assigned a specific bed in a ward. Conversely, the Ward entity has partial participation because an empty or newly opened ward can exist without housing any active patients. Similarly, the **Doctor–Patient** relationship is partial on both sides to accommodate a newly hired doctor who has not yet been assigned any active cases, or a patient who has just checked in and is undergoing initial triage before a doctor is assigned.

For attribute design, the patient's phone number was modeled as a multivalued attribute (drawn with a double oval) rather than a single-valued one. This accommodates real-world situations where a patient may provide both a personal cell number and an emergency contact number, preventing the data truncation or flat-file redundancy that occurs with restrictive single-value strings.